

Smoothing Methods in R

Zachary Tipton

2026-04-23

Smoothing methods

To estimate a rolling average, we will use the `slider` package. You may need to install it using `install.packages("slider")`.

We will experiment with the University of Arkansas enrollment data:

```
library(slider)
```

Warning: package 'slider' was built under R version 4.5.2

```
uark_enrollment <- data.frame(  
  year = c(2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024),  
  full_time = c(19508, 20379, 21047, 21415, 21668, 22144, 22602, 22193, 22070, 23282, 25214,  
  part_time = c(5029, 4962, 5190, 5339, 5526, 5414, 5176, 5366, 5492, 5786, 5722, 3714, 3724,  
)  
uark_enrollment$total <- uark_enrollment$full_time + uark_enrollment$part_time  
print(uark_enrollment$total)
```

```
[1] 24537 25341 26237 26754 27194 27558 27778 27559 27562 29068 30936 32140  
[13] 33610
```

The basic call is `slide_dbl(x, mean, .before = #, .after = #, .complete = TRUE)`

If we do `.before = 0` and `.after = 0`, we will average only y_t :

```
slide_dbl(uark_enrollment$total, mean, .before = 0, .after = 0, .complete = TRUE)
```

```
[1] 24537 25341 26237 26754 27194 27558 27778 27559 27562 29068 30936 32140
[13] 33610
```

If we want to average $\pm$ one year, we can use `.before = 1, .after = 1`

```
slide_dbl(uark_enrollment$total, mean, .before = 1, .after = 1, .complete = TRUE)
```

```
[1]      NA 25371.67 26110.67 26728.33 27168.67 27510.00 27631.67 27633.00
[9] 28063.00 29188.67 30714.67 32228.67      NA
```

Exercise

Manually, check this is true for y_{2013} by averaging the first 3 `uark_enrollment$total`:

```
enroll2013_smooth <- (uark_enrollment$total[1] + uark_enrollment$total[2] + uark_enrollment$total[3]) / 3
```

```
[1] 25371.67
```

Using last-period's observation

For forecasting into the future, the simplest method is to use last period's observation into the future. We already learned how to do this using `c(NA, uark_enrollment$total[1:(T - 1)])`, but we can do it more simply by using `.before = 1, .after = -1`:

```
uark_enrollment$total
```

```
[1] 24537 25341 26237 26754 27194 27558 27778 27559 27562 29068 30936 32140
[13] 33610
```

```
T <- nrow(uark_enrollment)
# y_{t-1}
c(NA, uark_enrollment$total[1:(T-1)])
```

```
[1]      NA 24537 25341 26237 26754 27194 27558 27778 27559 27562 29068 30936
[13] 32140
```

```
# Lag $y_t$ by 1
slide_dbl(uark_enrollment$total, mean, .before = 1, .after = -1, .complete = TRUE)
```

```
[1] NA 24537 25341 26237 26754 27194 27558 27778 27559 27562 29068 30936
[13] 32140
```

Last K period's average y_t

Or, we can do a rolling average of the previous values of y_{t-k} from $k = 0, \dots, K$

```
slide_dbl(uark_enrollment$total, mean, .before = 3, .after = 0, .complete = TRUE)
```

```
[1] NA NA NA 25717.25 26381.50 26935.75 27321.00 27522.25
[9] 27614.25 27991.75 28781.25 29926.50 31438.50
```

Exercise

1. Get y_{t+1} using `slide_dbl` and store this in a variable called `y_lead` in `uark_enrollment`
2. Calculate the ρ_1 autocorrelation coefficient using this newly created variable
3. Compare this to the result of `acf` function

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

```
filter, lag
```

The following objects are masked from 'package:base':

```
intersect, setdiff, setequal, union
```

```
library(slider)

# 1) Get y_{t+1} (lead) using slide_dbl
uark_enrollment <- uark_enrollment %>%
  mutate(
    y_lead = slide_dbl(
      total,
      mean,
      .before = -1,
      .after = 1,
      .complete = TRUE
    )
  )

# 2) rho_1 using the ACF formula
y <- uark_enrollment$total
y_bar <- mean(y)
rho1_manual <- sum((y[-length(y)] - y_bar) * (y[-1] - y_bar)) / sum((y - y_bar)^2)
rho1_manual
```

```
[1] 0.6729186
```

```
# 3) Compare to acf result
acf_result <- acf(y, plot = FALSE)
rho1_acf <- acf_result$acf[2]
rho1_acf
```

```
[1] 0.6729186
```

Election Polling

```
library(lubridate) # install.packages("lubridate")
```

Attaching package: 'lubridate'

The following objects are masked from 'package:base':

```
date, intersect, setdiff, union
```

```
library(slider) # install.packages("slider")
library(forecast) # install.packages("forecast")
```

Warning: package 'forecast' was built under R version 4.5.2

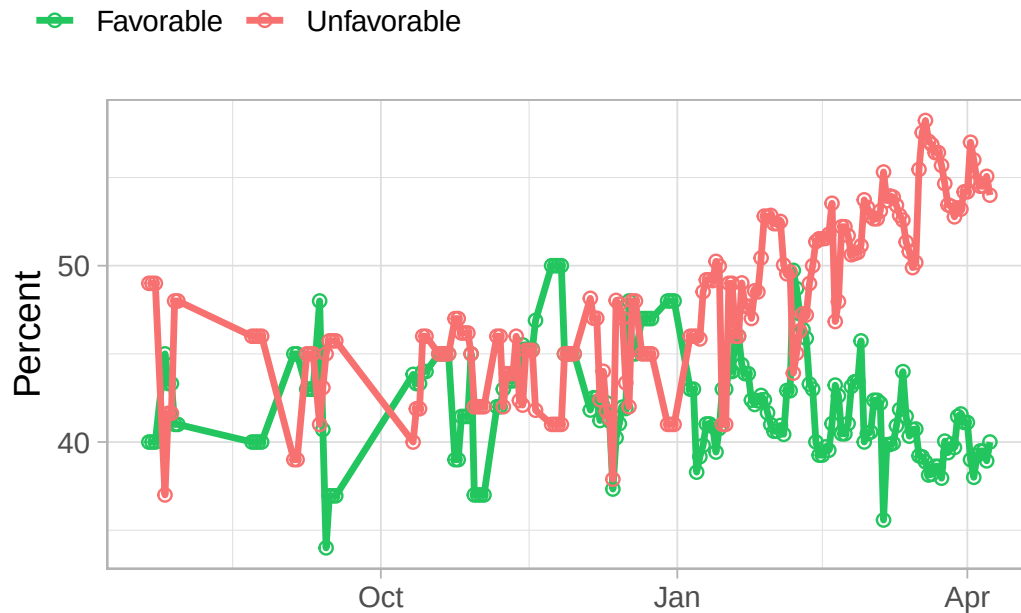
```
# Time-series data
avg_polls <- read.csv("data/silver_bulletin_musk_favorability.csv")
avg_polls$date <- ymd(avg_polls$date)
```

```
library(ggplot2) # install.packages("ggplot2")

# Custom function to make nice looking plots with raw polls and smoothed
plot_polls <- function(avg_polls, favorable_colname, unfavorable_colname) {
  ggplot() +
  geom_point(
    aes(x = date, y = favorable, color = "A"),
    data = avg_polls, shape = 21
  ) +
  geom_line(
    aes(x = date, y = .data[[favorable_colname]], color = "A"),
    data = avg_polls, size = 1.2
  ) +
  geom_point(
    aes(x = date, y = unfavorable, color = "B"),
    data = avg_polls, shape = 21
  ) +
  geom_line(
    aes(x = date, y = .data[[unfavorable_colname]], color = "B"),
    data = avg_polls, size = 1.2
  ) +
  scale_color_manual(
    values = c("A" = "#22c55e", "B" = "#f87171"),
    labels = c("A" = "Favorable", "B" = "Unfavorable")
  ) +
  labs(color = NULL, x = NULL, y = "Percent") +
  theme_light(base_size = 14) +
  theme(
    legend.position = "top",
    legend.justification = c(0, 1),
    legend.location = "plot"
  )
}
```

```
plot_polls(avg_polls, "favorable", "unfavorable")
```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.



Perform moving averages

The most common way this noisy data is summarized is with a moving average. Since we are trying to forecast the future election, let's using a moving average only using $y_{t-K}, \dots, y_t$ (i.e. `.before = K` and `.after = 0`).

Do the following:

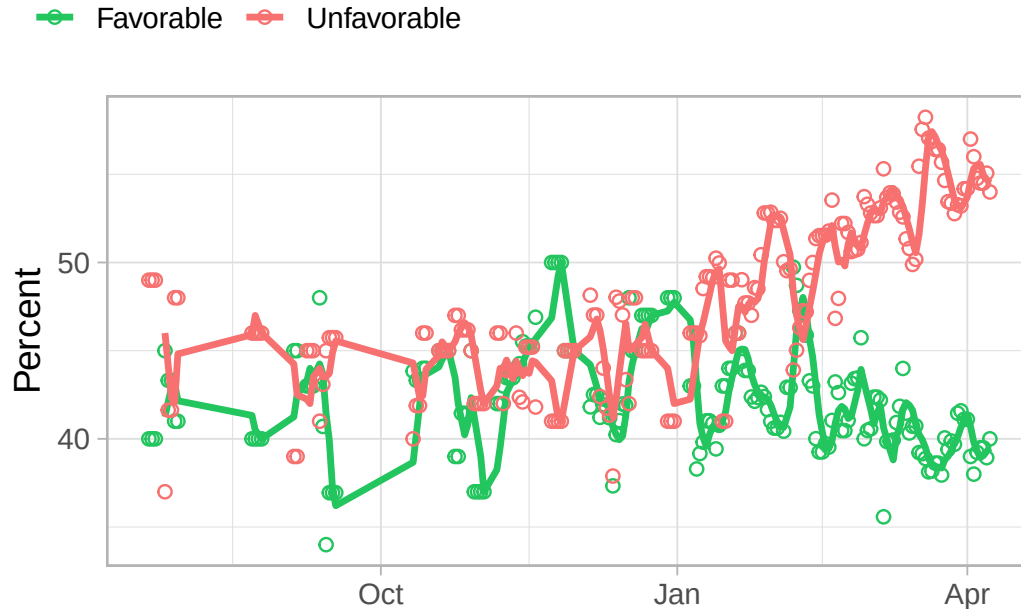
1. Create a 3-day one-sided moving average of `favorable` and `unfavorable`.

```
# Insert code here
avg_polls$favorable_ma_3day <- slide_dbl(avg_polls$favorable, mean, .before = 3, .after = 0,
avg_polls$unfavorable_ma_3day <- slide_dbl(avg_polls$unfavorable, mean, .before = 3, .after = 0)

# Then plot it using our function:
plot_polls(avg_polls, "favorable_ma_3day", "unfavorable_ma_3day")
```

Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_line()``).

Removed 3 rows containing missing values or values outside the scale range (``geom_line()``).



2. Create a 7-day one-sided moving average:

```
# Insert code here:
```

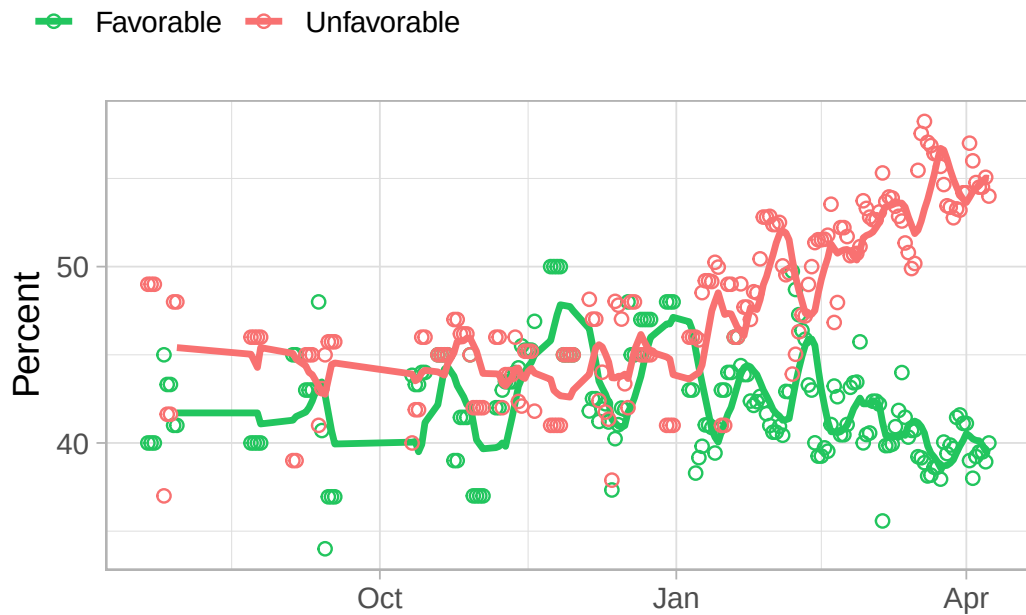
```
avg_polls$favorable_ma_7day <- slide_dbl(avg_polls$favorable, mean, .before = 7, .after = 0,  
avg_polls$unfavorable_ma_7day <- slide_dbl(avg_polls$unfavorable, mean, .before = 7, .after = 0)
```

```
# Then plot it using our function:
```

```
plot_polls(avg_polls, "favorable_ma_7day", "unfavorable_ma_7day")
```

Warning: Removed 7 rows containing missing values or values outside the scale range (``geom_line()``).

Removed 7 rows containing missing values or values outside the scale range (``geom_line()``).



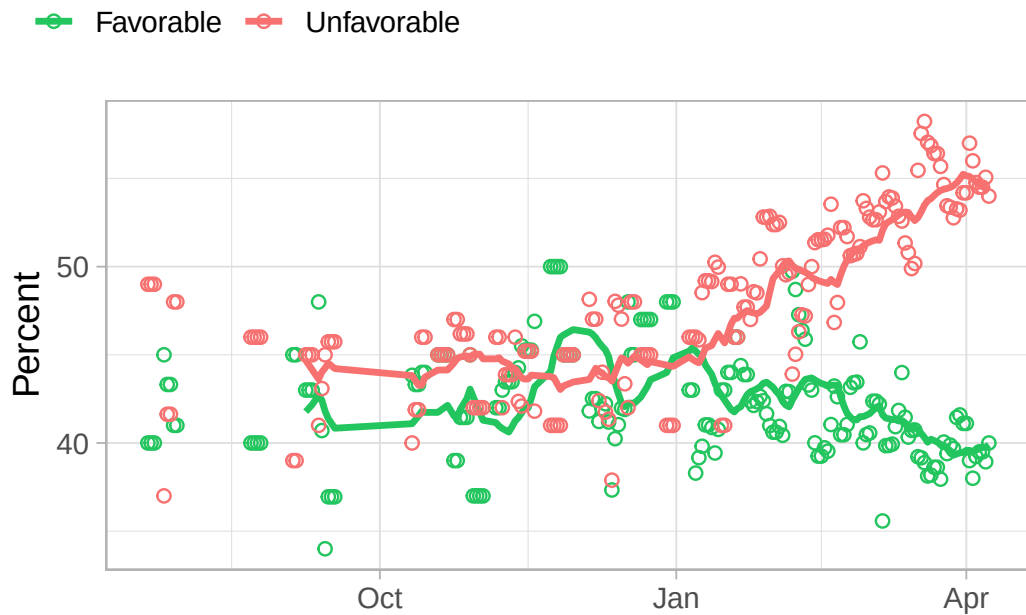
3. Last, create a 14-day one-sided moving average:

```
# Insert code here:
avg_polls$favorable_ma_14day <- slide_dbl(avg_polls$favorable, mean, .before = 14, .after = 0)
avg_polls$unfavorable_ma_14day <- slide_dbl(avg_polls$unfavorable, mean, .before = 14, .after = 0)

# Then plot it using our function:
plot_polls(avg_polls, "favorable_ma_14day", "unfavorable_ma_14day")
```

Warning: Removed 14 rows containing missing values or values outside the scale range (``geom_line()``).

Removed 14 rows containing missing values or values outside the scale range (``geom_line()``).



Interpretation

Compare the 3-day moving average versus the 14-day moving average. Give a reason for why you might prefer the 3-day moving average and a reason for why you might prefer the 14-day moving average.

Answer:

The 3-day moving average shows an interesting pattern of divergence and convergence between favorable and unfavorable. There's almost a steady rhythm to it. I'm curious if that's an artifact of poll timing, or if there is some news cycle thing behind it.

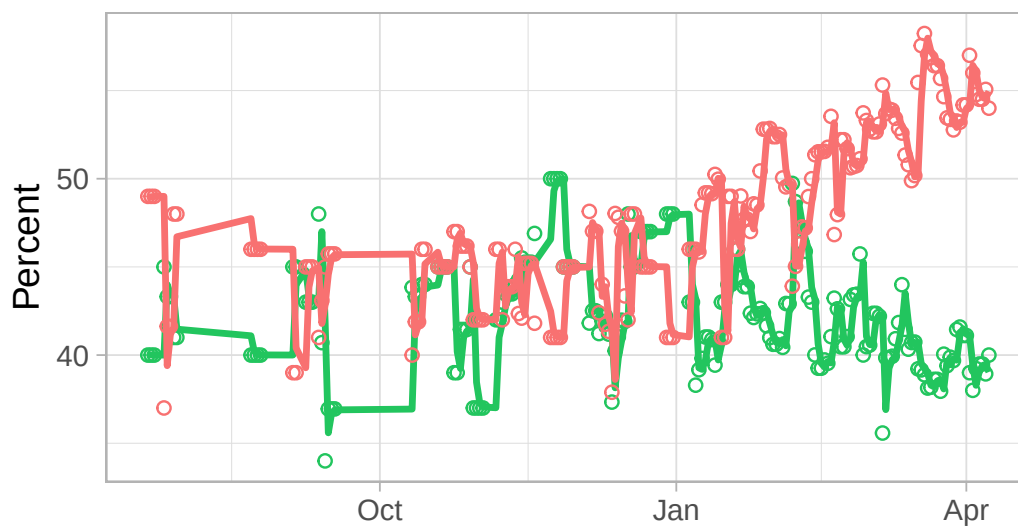
The 14-day moving average is much easier to interpret and narrows big changes to specific areas of interest. Like what's going on in December where there is a large bump in favorability? It's also a lot easier to see where things go downhill in terms of favorability. From some domain knowledge I know that a lot of these reporters (Nate Silver, Galen Druke, etc. RIP 538) look at polling averages to try and find the signal in the noise. Polls are noisy things.

Simple exponential smoothing

Here is code to perform simple exponential smoothing. `alpha` is the parameter that determines the smoothness:

```
# `fitted` grabs the fitted values in the observed time-series
avg_polls$favorable_ses_pt8 = ses(avg_polls$favorable, alpha = 0.8)$fitted
avg_polls$unfavorable_ses_pt8 = ses(avg_polls$unfavorable, alpha = 0.8)$fitted
plot_polls(avg_polls, "favorable_ses_pt8", "unfavorable_ses_pt8")
```

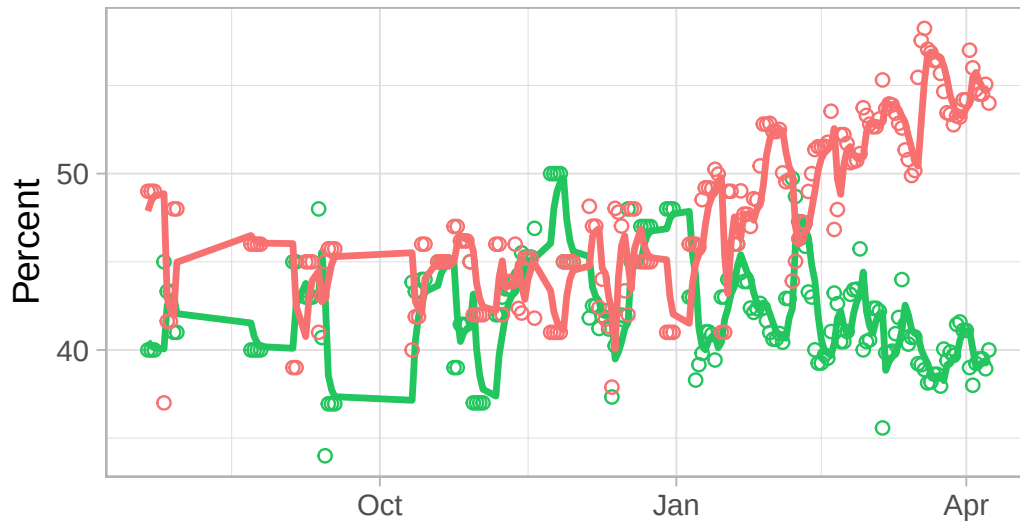
—○— Favorable —○— Unfavorable



Try now with $\alpha = 0.5$ and $\alpha = 0.2$

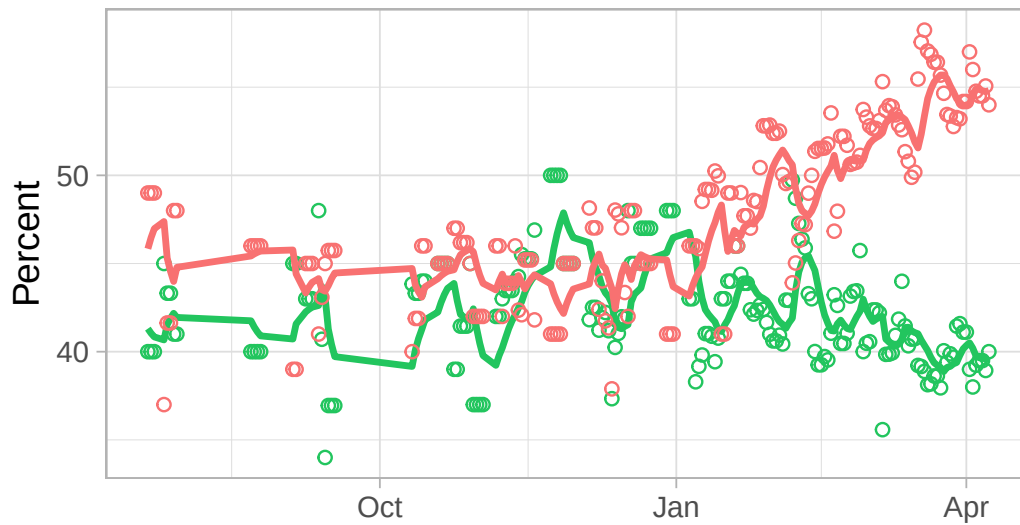
```
avg_polls$favorable_ses_pt5 = ses(avg_polls$favorable, alpha = 0.5)$fitted
avg_polls$unfavorable_ses_pt5 = ses(avg_polls$unfavorable, alpha = 0.5)$fitted
plot_polls(avg_polls, "favorable_ses_pt5", "unfavorable_ses_pt5")
```

—○— Favorable —○— Unfavorable



```
avg_polls$favorable_ses_pt2 = ses(avg_polls$favorable, alpha = 0.2)$fitted
avg_polls$unfavorable_ses_pt2 = ses(avg_polls$unfavorable, alpha = 0.2)$fitted
plot_polls(avg_polls, "favorable_ses_pt2", "unfavorable_ses_pt2")
```

—○— Favorable —○— Unfavorable



Interpretation

Write answers to the following questions:

1. In your own words, what does the alpha parameter change about our prediction?

Answer:

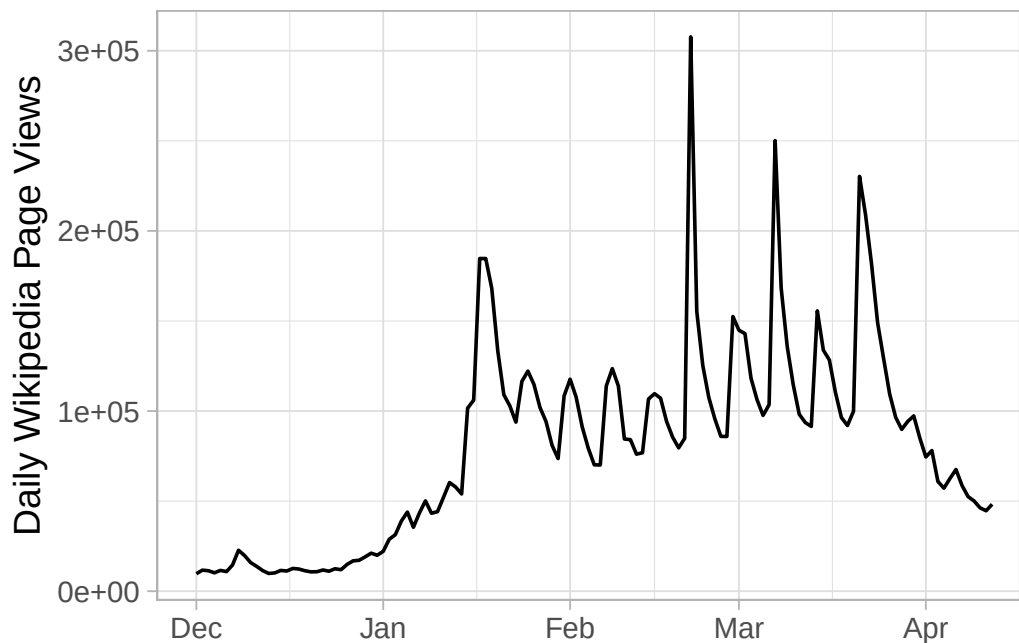
Alpha changes how much noise the model is chasing.

Severance popularity

Now, for the final part of this assignment, we will be looking at daily wikipedia views for the TV show Severance.

```
df <- read.csv("data/severance_wikipedia_views.csv")
# convert to `Date` class
df$date <- ymd(df$date)
# Filter since December 2024
df <- df[df$date >= ymd("2024-12-01"), ]

ggplot() +
  geom_line(aes(x = date, y = views), data = df) +
  labs(x = NULL, y = "Daily Wikipedia Page Views") +
  theme_light(base_size = 14)
```



Here we are going to try and conduct inference on this time-series. We will try two approaches:

1. A one-sided 5-day moving average
2. Simple exponential smoothing

Please comment on problems that each one has at capturing the spikes on the Friday premiere days

```
df$views_5day_ma <- slide_dbl(df$views, mean, .before = 4L, .after = 0L, .complete = TRUE)
df$views_ses <- ses(df$views)$fitted
summary(ses(df$views))
```

Forecast method: Simple exponential smoothing

Model Information:

Simple exponential smoothing

Call:

```
ses(y = df$views)
```

Smoothing parameters:

alpha = 0.6573

Initial states:

l = 10288.7861

sigma: 33307.82

	AIC	AICc	BIC
	3424.405	3424.591	3433.076

Error measures:

	ME	RMSE	MAE	MPE	MAPE	MASE	ACF1
Training set	424.8363	33056.44	17008.81	-2.000482	16.72433	1.08865	0.07898528

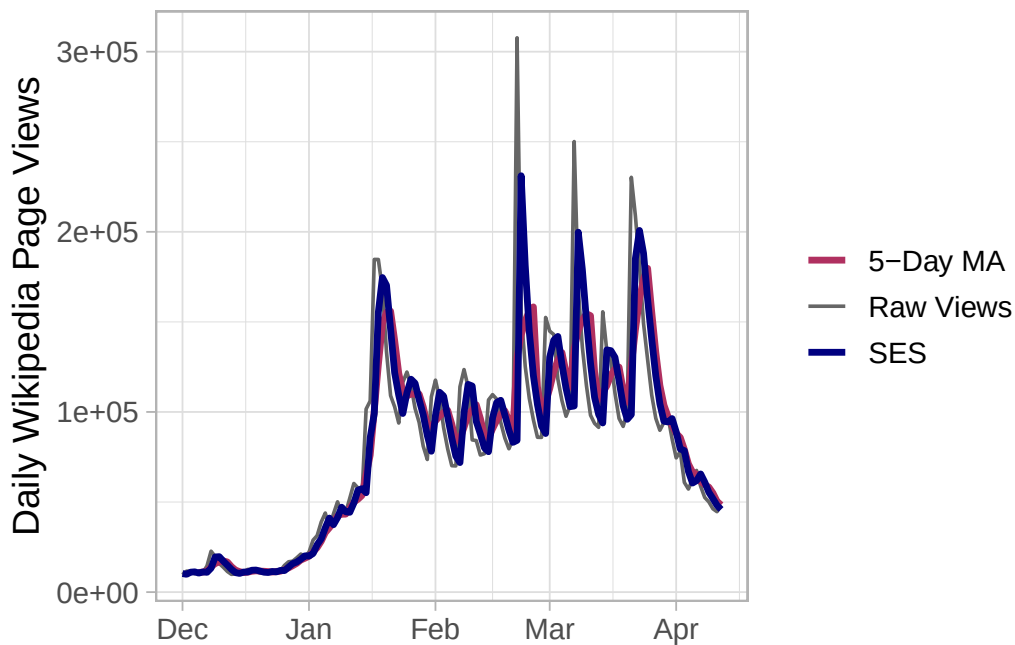
Forecasts:

	Point Forecast	Lo 80	Hi 80	Lo 95	Hi 95
134	47427.59	4741.896	90113.28	-17854.54	112709.7
135	47427.59	-3653.209	98508.38	-30693.75	125548.9
136	47427.59	-10851.289	105706.46	-41702.26	136557.4
137	47427.59	-17253.221	112108.40	-51493.17	146348.3
138	47427.59	-23076.218	117931.39	-60398.68	155253.9
139	47427.59	-28453.676	123308.85	-68622.79	163478.0
140	47427.59	-33474.488	128329.66	-76301.46	171156.6
141	47427.59	-38201.411	133056.59	-83530.66	178385.8
142	47427.59	-42680.708	137535.88	-90381.16	185236.3

143 47427.59 -46947.645 141802.82 -96906.88 191762.1

```
ggplot() +  
  geom_line(aes(x = date, y = views, color = "Raw Views"), data = df) +  
  geom_line(aes(x = date, y = views_5day_ma, color = "5-Day MA"),  
            data = df, linewidth = 1.2) +  
  geom_line(aes(x = date, y = views_ses, color = "SES"),  
            data = df, linewidth = 1.2) +  
  scale_color_manual(  
    values = c("Raw Views" = "grey40", "5-Day MA" = "maroon", "SES" = "navy")  
  ) +  
  labs(x = NULL, y = "Daily Wikipedia Page Views", color = NULL) +  
  theme_light(base_size = 14)
```

Warning: Removed 4 rows containing missing values or values outside the scale range (`geom\_line()`).



Answer:

Well both of them are no longer on Friday - so that's an issue. The 5-day MA sort of "builds up" to the Friday premiere, which doesn't make much sense. In this particular situation I don't think we can assume that viewship on Monday has that much relationship to viewship on Friday (if it is a daily show). I'm sure there is some relationship, but having this like half-dome shape in the 5-day MA is inaccurate. SES does a much better job but has the same issue of undershooting and being off temporally.